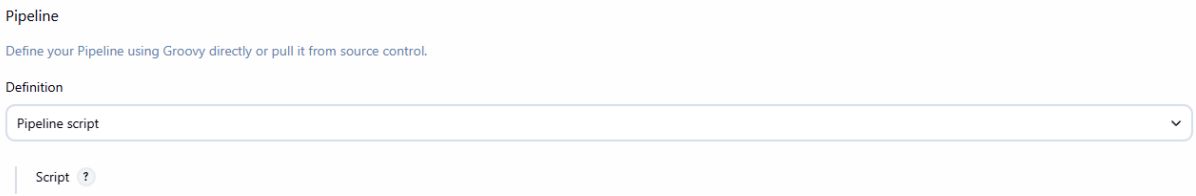
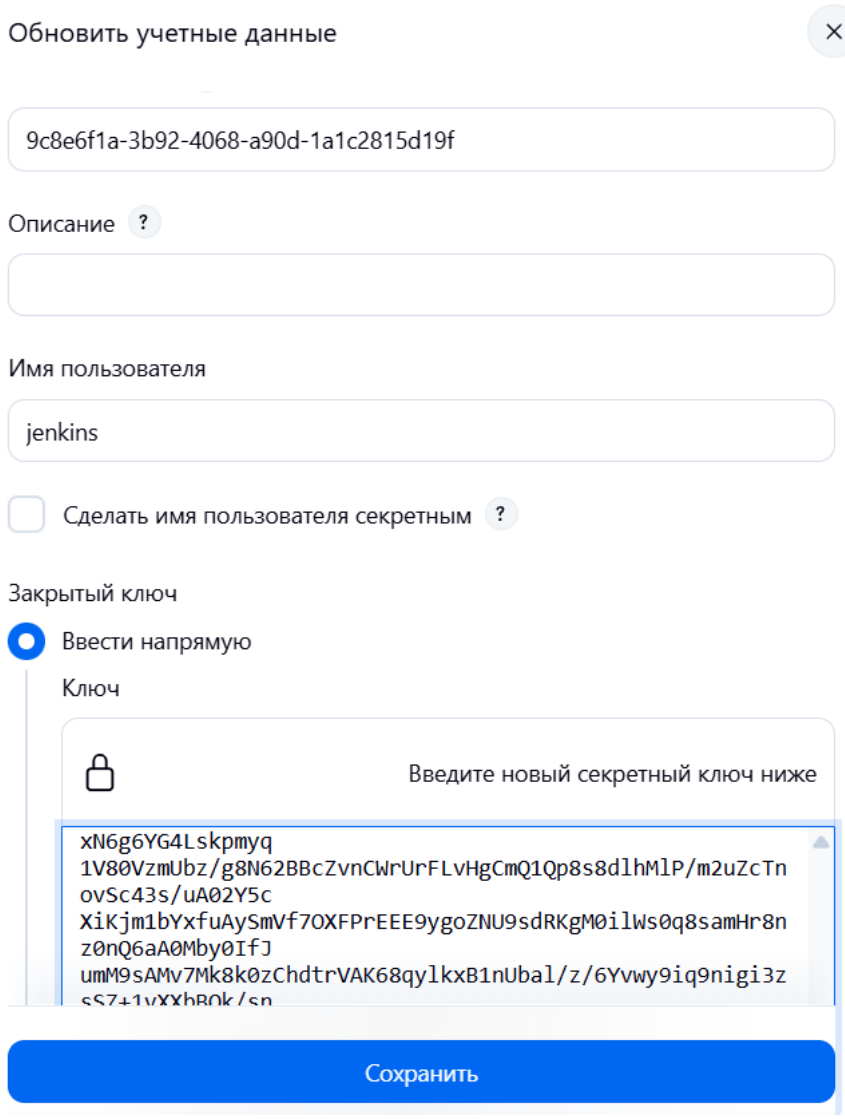
1. Создаю новый item с именем HW-30 и типом Pipeline:



2. Перехожу в конфигурации созданного item, спускаюсь до пункта Pipeline и выбираю Pipeline script:



3. У меня есть мини проект с прошлой ДЗ, там есть готовый дженкинс файл, но для выполнения этого ДЗ, я удалю его и создам свой
4. Для написания pipeline добавляю креды для подключения к удаленному серверу по ssh ключу. Подключаться буду к серверу 62.60.246.108.



5. При запуске возникла такая ошибка: + git clone https://github.com/SeigneurPuppy1/jenkins / фатальный: путь назначения 'jenkins' уже существует и не является пустым каталогом. Чтобы исправить ошибку, необходимо зайти на сервер дженкинс и перейти по пути /var/lib/Jenkins/workspace/

```
anastasia@seigneur-puppy:/var/lib/jenkins/workspace$ ll
total 40
drwxr-xr-x 10 jenkins jenkins 4096 Jul 24 19:22  ./
drwxr-xr-x 19 jenkins jenkins 4096 Jul 25 15:42  ../
drwxr-xr-x  6 jenkins jenkins 4096 Jul 19 00:32  first/
drwxr-xr-x  3 jenkins jenkins 4096 Jul 23 18:11  HW-30/
drwxr-xr-x  2 jenkins jenkins 4096 Jul 25 15:41 'HW-30@tmp'/
drwxr-xr-x  3 jenkins jenkins 4096 Jul 22 18:10  learn-groovy/
drwxr-xr-x  2 jenkins jenkins 4096 Jul 23 13:42 'learn-groovy@tmp'/
drwxr-xr-x  4 jenkins jenkins 4096 Jul 24 19:22  report-main/
drwxr-xr-x  2 jenkins jenkins 4096 Jul 24 19:22 'report-main@tmp'/
drwxr-xr-x  2 jenkins jenkins 4096 Jul 20 12:18  report-test/
```

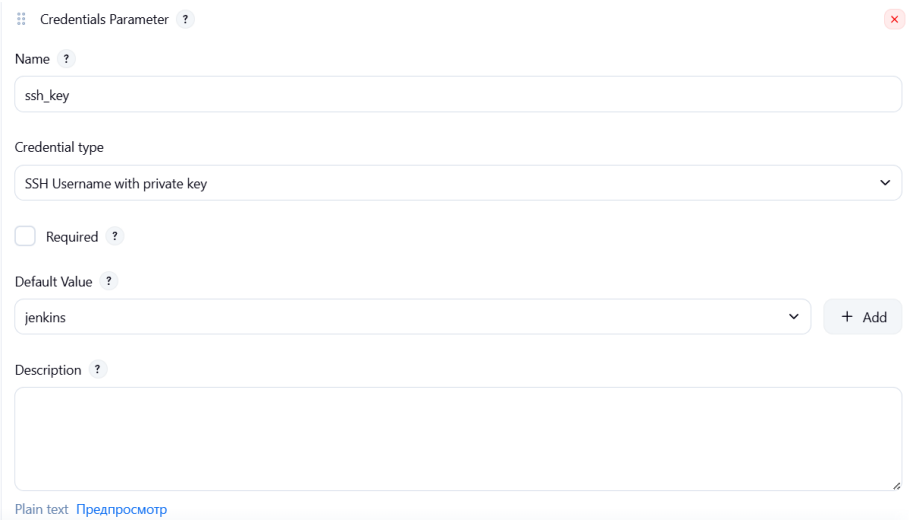
Здесь мы видим все созданные item, тот в котором я произвожу настройку пайплана называется HW-30 там и есть папка jenkins в которой уже существует клонируемый нами репозиторий. Необходимо его удалить, чтобы джоба отработала, либо так как я постоянно перезапускаю и тестирую отработку добавить сразу в пайплайн пункт удадения созданной директории Jenkins, тогда каждый запуск будет начинаться с чистого листа

```
anastasia@seigneur-puppy:/var/lib/jenkins/workspace/HW-30$ ll
total 12
drwxr-xr-x  3 jenkins jenkins 4096 Jul 23 18:11 ./
drwxr-xr-x 10 jenkins jenkins 4096 Jul 24 19:22 ../
drwxr-xr-x  4 jenkins jenkins 4096 Jul 23 18:11 jenkins/
anastasia@seigneur-puppy:/var/lib/jenkins/workspace/HW-30$ ll jenkins/
total 20
drwxr-xr-x 4 jenkins jenkins 4096 Jul 23 18:11 ./
drwxr-xr-x 3 jenkins jenkins 4096 Jul 23 18:11 ../
drwxr-xr-x 8 jenkins jenkins 4096 Jul 23 18:11 .git/
-rw-r--r-- 1 jenkins jenkins    8 Jul 23 18:11 .gitignore
-rw-r--r-- 1 jenkins jenkins    0 Jul 23 18:11 README.md
drwxr-xr-x 4 jenkins jenkins 4096 Jul 23 18:11 simple-java-maven-app/
anastasia@seigneur-puppy:/var/lib/jenkins/workspace/HW-30$
```

```
stage('Build') {
    steps {
        sh """
        rm -rf jenkins
```

6. Добавляю в пайплайн пункт с sshagent (но перед этим проверяем что этот плагин установлен, в веб интерфейсе дженкинс), но этот параметр должен находиться в оболочке script {…}. Также я указала параметризированную сборку ssh_key, чтобы при запуске он доставал настроенный ssh ключ для подключения к серверу:

```
stage('Deploy'){
   steps {
       script {
           sshagent(credentials: ['ssh_key']) {
               sh """
               ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null little@"${ip}"
               """
       }
```



7. При запуске пайплайна с такой структурой, отрабатывается успешно, теперь преходим к наполнению скрипта:

```
pipeline {
    agent any

    stages{
        stage('Checkout') {
            steps {
                sh """ping -c 4 ${ip}"""
            }
        }
        stage('Build') {
            steps {
                sh """
                rm -rf jenkins
                git clone 'https://github.com/SeigneurPuppy1/jenkins.git'
                cd ./jenkins/simple-java-maven-app
                mvn clean compile
                """
            }
        }
        stage('Test') {
            steps {
                sh """
                cd ./jenkins/simple-java-maven-app
                mvn clean test
                """
            }
        }
         stage('Deploy'){
            steps {
                script {
                    sshagent(credentials: ['ssh_key']) {
                        sh """
                        ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null little@"${ip}"
                        """
                }
            }
         }
      }
   }
}
```

8. Добавлю объект params, описывать в пайплане его нужно после строки agent any. Например, до этого мы добавляли параметр ip с помощью пункта «параметризированная сборка» и там указывали в строках необходимые значения, также добавила параметр наименования нашего приложения, пока что, в дальнейшем скорее всего будет добавлены еще параметры. Эти же значения можно описать в самом пайплайн:

```
pipeline {
    agent any
    parameters {
        string(name: 'server', defaultValue: '62.60.246.108', description: 'Сервер для запуска приложения')
        string(name: 'APP_NAME', defaultValue: 'maven-app', description: 'Название приложения')
    }
```

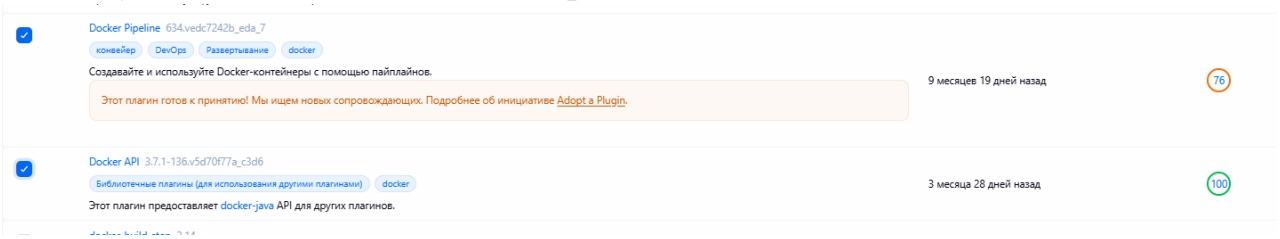
9. Также добавляю переменные для удобочитаемости в дальнейшем и если захочу что-то изменить, то не придется искать указанные значения по всему коду. env.WORKSPACE системная переменная дженкинса, которая содержит в себе путь до нашего проекта то есть /var/lib/Jenkins/workspace/HW-30/

```
environment {
    WORKSPACE_DIR = "${env.WORKSPACE}"
    APP_DIR = 'simple-java-maven-app'
    JAR_FILE = "${APP_NAME}.jar"
}
```

10. Теперь добавим условие when. Добавила условие в этап Test, связанное с проверкой ветки. То есть если ветка main, то выполнится шаг тестирования, если нет, то он пропустит этот шаг и пайплайн упадет. У меня ветка называется jenkins, но для теста я указала main и запустила. Вот итог, джоба споткнулась об условие и завершила дальнейшие действия:

```
stage('Test') {
      when {
          expression {
           return params.BRANCH == 'jenkins'
          }
          }
```

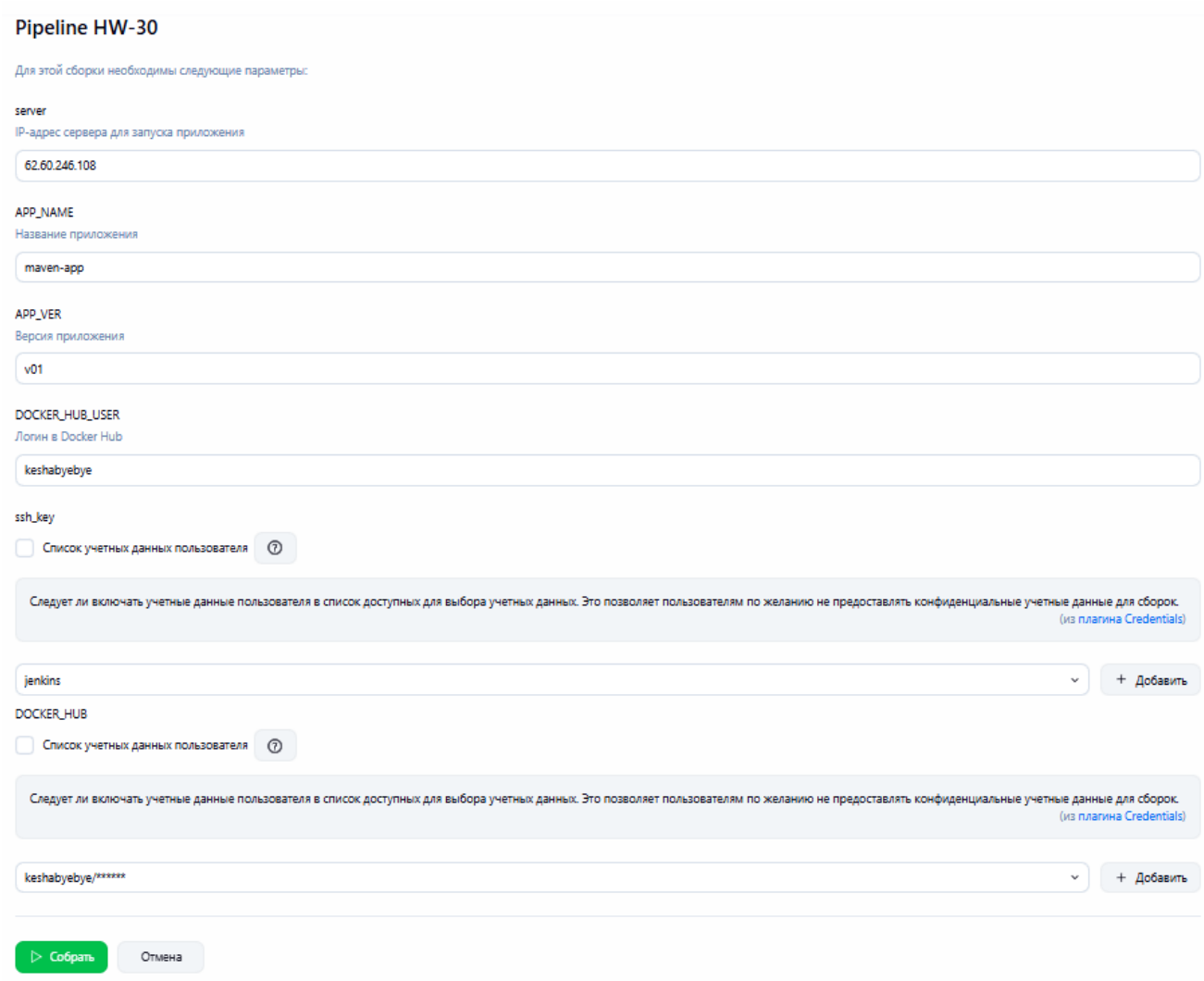
11. Итак, то что было описано выше, это примерно первые 5-6 пунктов из задания. Далее пришлось его наращивать, с учетом сборки через docker контейнера. Для начала устанавливаю плагины докера в Jenkins:



12. Начинаю с добавление еще параметров, которые будут использоваться дальше в пайплайн. У меня не получилось запустить контейнер, точнее скопировать его напрямую, поэтому был выбран вариант, чтобы сформировать докер образ и отправить его на Docker Hub, после чего вытащить этот образ с сайта и запустить контейнер на целевом хосте. Эти параметры обычно указываются при запуске джобы:

```
pipeline {
    agent any

    parameters {
        string(name: 'server', defaultValue: '62.60.246.108', description: 'IP сервера для запуска приложения')
        string(name: 'APP_NAME', defaultValue: 'maven-app', description: 'Название приложения')
        string(name: 'APP_VER', defaultValue: 'v01', description: 'Версия приложения')
        string(name: 'DOCKER_HUB_USER', defaultValue: 'keshabyebye', description: 'Логин на Docker Hub')
    }
```



13. Задаем переменные окружения. Как можно увидеть, у дженкинса есть дефолтная переменная вида env.WORKSPACE, о ней было сказано ранее, в пункте 9. APP_DIR, это указание директории в которой будет лежать наш код. DOCKER_IMAGE по какому пути и какое название будет у собранного образа на докер хаб

```
environment {
    WORKSPACE_DIR = "${env.WORKSPACE}"
    APP_DIR = 'simple-java-maven-app'
    DOCKER_IMAGE = "${params.DOCKER_HUB_USER}/${params.APP_NAME}:${params.APP_VER}"
}
```

14. Сначала проверяем, что целевой сервер доступен с помощью простого ping. Также как можно увидеть мы взяли наименование server из параметров заданных выше, чтобы взять значенит параметра мы записываем его как params.server

```
stages {
    stage('Pre-check') {
        steps {
            script {
                echo "Проверяем доступность сервера ${params.server}..."
                sh "ping -c 2 ${params.server} || echo 'Ping не прошел, но пробуем дальше'"
            }
        }
    }
```

```
Проверяем доступность сервера 62.60.246.108...
[Pipeline] sh
+ ping -c 2 62.60.246.108
PING 62.60.246.108 (62.60.246.108) 56(84) bytes of data.
64 bytes from 62.60.246.108: icmp_seq=1 ttl=59 time=39.2 ms
64 bytes from 62.60.246.108: icmp_seq=2 ttl=59 time=39.2 ms

--- 62.60.246.108 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 39.183/39.185/39.188/0.002 ms
```

15. Далее отдельным шагом мы клонируем репозиторий, но перед этим удаляем старую директорию, чтобы не было конфликтов, что такая директория существует, так как тест пайплайна будет неоднократным. После чего клонируем из определенной ветки, в моем случае ‘jenkins’:

```
    stage('Checkout') {
        steps {
            script {
                echo "Клонируем репозиторий..."
                sh "rm -rf ${APP_DIR}"
                git branch: 'jenkins', url: 'https://github.com/SeigneurPuppy1/jenkins.git'
            }
        }
    }
```

```
Клонируем репозиторий...
[Pipeline] sh
+ rm -rf simple-java-maven-app
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/SeigneurPuppy1/jenkins.git
 > git init /var/lib/jenkins/workspace/HW-30 # timeout=10
Получение изменений из вышестоящего репозитория https://github.com/SeigneurPuppy1/jenkins.git
 > git --version # timeout=10
 > git --version # 'git version 2.43.0'
 > git fetch --tags --force --progress -- https://github.com/SeigneurPuppy1/jenkins.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/SeigneurPuppy1/jenkins.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Избегайте повторной выборки
 > git rev-parse refs/remotes/origin/jenkins^{commit} # timeout=10
Проверка ревизии 6d421e361085edebffb89ada7a599ec864333ed6 (refs/remotes/origin/jenkins)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 6d421e361085edebffb89ada7a599ec864333ed6 # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git checkout -b jenkins 6d421e361085edebffb89ada7a599ec864333ed6 # timeout=10
Сообщение о коммите: «Удаление Jenkinsfile»
```

16. Собираем наш проект с помощью maven. Сборку, тест, формирование докер образа происходят на тачке, на которой запущен дженкинс. И только развертывание приложения происходит на целевом хосте. Итак, здесь использовался метод перехода в папку с более простой логикой и без нагромождений, так как для обычного sh “cd …” пришлось бы ставить знаки переноса строк, чтобы он не просто выполнил первую команду и закрылся, а чтобы выполнил все указанные. Так вот метод dir() помогает с этим. Указываем директорию нашего клонированного проекта. Соответственно далее запускаем сборку maven и показываем созданный .jar по пути, который дефолтный target/

```
stage('Build') {
    steps {
        dir("${APP_DIR}") {
            sh """
            echo "=== СБОРКА MAVEN ПРОЕКТА ==="
            mvn clean package
            echo "JAR файлы:"
            ls -la target/*.jar
            """
        }
    }
}
```

```
Выполнение в /var/lib/jenkins/workspace/HW-30/simple-java-maven-app
[Pipeline] {
[Pipeline] sh
+ echo === СБОРКА ПРОЕКТА MAVEN ===
=== СБОРКА ПРОЕКТА MAVEN ===
+ mvn clean package
[ [1;34mINFO [m] Сканирование проектов...
```

```
[␛[1;34mINFO␛[m] -------------------------------------------------------
[␛[1;34mINFO␛[m]  T E S T S
[␛[1;34mINFO␛[m] -------------------------------------------------------
[␛[1;34mINFO␛[m] Running com.mycompany.app.␛[1mAppTest␛[m
[␛[1;34mINFO␛[m] ␛[1;32mTests run: ␛[0;1;32m2␛[m, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.104 s - in com.mycompany.app.␛[1mAppTest␛[m
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] Results:
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1;32mTests run: 2, Failures: 0, Errors: 0, Skipped: 0␛[m
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1m--- ␛[0;32mmaven-jar-plugin:3.5.0:jar␛[m ␛[1m(default-jar)␛[m @ ␛[36mmy-
app␛[0;1m ---␛[m
[␛[1;34mINFO␛[m] Building jar: /var/lib/jenkins/workspace/HW-30/simple-java-maven-app/target/my-app-
1.0-SNAPSHOT.jar
[␛[1;34mINFO␛[m] ␛[1m------------------------------------------------------------------------␛[m
[␛[1;34mINFO␛[m] ␛[1;32mBUILD SUCCESS␛[m
[␛[1;34mINFO␛[m] ␛[1m------------------------------------------------------------------------␛[m
[␛[1;34mINFO␛[m] Total time:  3.742 s
[␛[1;34mINFO␛[m] Finished at: 2026-08-03T12:16:05+03:00
[␛[1;34mINFO␛[m] ␛[1m------------------------------------------------------------------------␛[m
+ echo JAR файлы:
JAR файлы:
+ ls -la target/my-app-1.0-SNAPSHOT.jar
-rw-r--r-- 1 jenkins jenkins 2770 Aug  3 12:16 target/my-app-1.0-SNAPSHOT.jar
```

17. Далее тестируем наше собранный код, используем здесь конструкцию try-catch, то есть если запуск теста происходит с ошибкой то блок catch перехватывает их, выводит сообщение и вызывает error(), который немедленно останавливает пайплайн, чтобы сломанный код не был задеплоен:

```
stage('Test') {
    steps {
        script {
            try {
                echo "Запуск тестов..."
                dir("${APP_DIR}") {
                    sh "mvn test"
                }
                echo "Тесты пройдены"
            } catch (Exception e) {
                echo "Тесты не пройдены: ${e.message}"
                error("Тесты не пройдены")
            }
        }
    }
}
```

```
Запуск тестов...
[Pipeline] dir
Running in /var/lib/jenkins/workspace/HW-30/simple-java-maven-app
[Pipeline] {
[Pipeline] sh
+ mvn test
[␛[1;34mINFO␛[m] Scanning for projects...
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1m-----------------------< ␛[0;36mcom.mycompany.app:my-app␛[0;1m >----------------
------␛[m
[␛[1;34mINFO␛[m] ␛[1mBuilding my-app 1.0-SNAPSHOT␛[m
[␛[1;34mINFO␛[m] ␛[1m--------------------------------[ jar ]---------------------------------␛[m
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1m--- ␛[0;32mmaven-enforcer-plugin:3.6.3:enforce␛[m ␛[1m(enforce-maven)␛[m @
␛[36mmy-app␛[0;1m ---␛[m
[␛[1;34mINFO␛[m] Rule 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion passed
[␛[1;34mINFO␛[m] Rule 1: org.apache.maven.enforcer.rules.version.RequireJavaVersion passed
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1m--- ␛[0;32mmaven-resources-plugin:2.6:resources␛[m ␛[1m(default-resources)␛[m @
␛[36mmy-app␛[0;1m ---␛[m
[␛[1;34mINFO␛[m] Using 'UTF-8' encoding to copy filtered resources.
[␛[1;34mINFO␛[m] skip non existing resourceDirectory /var/lib/jenkins/workspace/HW-30/simple-java-
maven-app/src/main/resources
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1m--- ␛[0;32mmaven-compiler-plugin:3.15.0:compile␛[m ␛[1m(default-compile)␛[m @
␛[36mmy-app␛[0;1m ---␛[m
[␛[1;34mINFO␛[m] Nothing to compile - all classes are up to date.
[␛[1;34mINFO␛[m]
[␛[1;34mINFO␛[m] ␛[1m--- ␛[0;32mmaven-resources-plugin:2.6:testResources␛[m ␛[1m(default-
testResources)␛[m @ ␛[36mmy-app␛[0;1m ---␛[m
[␛[1;34mINFO␛[m] Using 'UTF-8' encoding to copy filtered resources.
[␛[1;34mINFO␛[m] skip non existing resourceDirectory /var/lib/jenkins/workspace/HW-30/simple-java-
[␛[1;34mINFO␛[m] ␛[1;32mBUILD SUCCESS␛[m
[␛[1;34mINFO␛[m] ␛[1m------------------------------------
[␛[1;34mINFO␛[m] Total time:  2.230 s
[␛[1;34mINFO␛[m] Finished at: 2026-08-03T12:16:09+03:00
[␛[1;34mINFO␛[m] ␛[1m------------------------------------
[Pipeline] }
[Pipeline] // dir
[Pipeline] echo
Тесты пройдены
```

18. Далее создаем докер файл с помощью writeFile (встроенная функция jenkins), то есть мы указываем в какой файл будет записано описание нашего контейнера, что в нем должно быть. Затем происходит сборка образа докера и соответственно отображает нам оп завершению, что образ точно создан

```
        stage('Docker Build') {
            steps {
                script {
                    echo "Создаем Docker образ..."
                    dir("${APP_DIR}") {
                        writeFile file: 'Dockerfile', text: """FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
"""

                        echo "Собираем образ: ${DOCKER_IMAGE}"
                        sh """
                        docker build -t ${DOCKER_IMAGE} .
                        echo "Проверка образа:"
                        docker images | grep ${params.APP_NAME}
                        """
                    }
                }
            }
        }
```

```
Создаем Docker образ...
[Pipeline] dir
Running in /var/lib/jenkins/workspace/HW-30/simple-java-maven-app
[Pipeline] {
[Pipeline] writeFile
[Pipeline] echo
Собираем образ: keshabyebye/maven-app:v01
[Pipeline] sh
+ docker build -t keshabyebye/maven-app:v01 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon     64kB

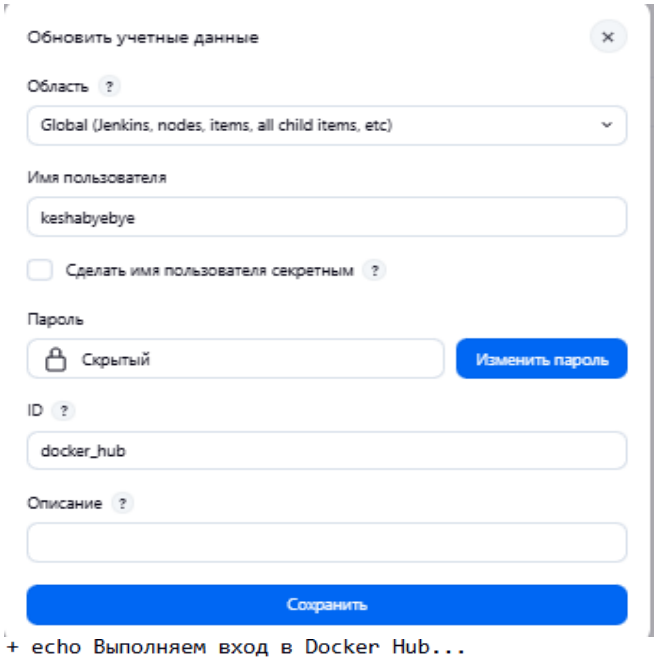
Step 1/5 : FROM eclipse-temurin:21-jre
 ---> 273396ed5998
Step 2/5 : WORKDIR /app
 ---> Using cache
 ---> 2ac4dce300fa
Step 3/5 : COPY target/*.jar app.jar
 ---> 613eed66ef55
Step 4/5 : EXPOSE 8080
 ---> Running in 696ef491f49a
 ---> Removed intermediate container 696ef491f49a
 ---> d621e33c9e1b
Step 5/5 : ENTRYPOINT ["java", "-jar", "app.jar"]
 ---> Running in 50cefb8b88c5
 ---> Removed intermediate container 50cefb8b88c5
 ---> 65343e286089
Successfully built 65343e286089
Successfully tagged keshabyebye/maven-app:v01
+ echo Проверка образа:
Проверка образа:
+ docker images
+ grep maven-app
WARNING: This output is designed for human readability. For machine-readable output, please use --
format.
keshabyebye/maven-app:v01   65343e286089        454MB          115MB
```

19. Следующее действие — это запушить наш готовый докер образ в Docker Hub. Для этого использую withCredentials, который временно выдает доступ для того, чтобы мы безопасно могли взять логин (DH_USER) и пароль (DH_PASS) из созданного мной Credentials в настройках дженкинс. Как только мы безопасно получили данные, авторизовываемся и пушим туда образ. После этого логин и пароль полностью очищаются

```
stage('Docker Push to Hub') {
    steps {
        script {
            echo "Загружаем образ на Docker Hub..."
            withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB', usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                sh """
                echo "Выполняем вход в Docker Hub..."
                echo "${DH_PASS}" | docker login -u "${DH_USER}" --password-stdin

                echo "Пушим образ ${DOCKER_IMAGE}..."
                docker push ${DOCKER_IMAGE}

                echo "Образ успешно загружен на Docker Hub!"
                """
            }
        }
    }
}
```




```
+ echo Выполняем вход в Docker Hub...
Выполняем вход в Docker Hub...
+ echo ****
+ docker login -u keshabyebye --password-stdin
Login Succeeded
+ echo Пушим образ keshabyebye/maven-app:v01...
Пушим образ keshabyebye/maven-app:v01...
+ docker push keshabyebye/maven-app:v01
The push refers to repository [docker.io/keshabyebye/maven-app]
ed819469700f: Waiting
 6fe36b853c88: Pushed
 v01: digest: sha256:65343e28608936470164a4fa6591169dbfe7d2238cc183919ee37998c5766841 size: 2047
 + echo Образ успешно загружен на Docker Hub!
 Образ успешно загружен на Docker Hub!
```

20. Самый большой кусок пайплайна. Деплоим наш готовый докер образ на целевой сервер, также создаем безопасную среду для извлечения логина и пароля для авторизации на докер хаб. Затем подключаемся к целевому серверу по ssh, используя созданные креды подключения по ssh в Credentials настройках Jenkins. При скачивании образа поставили проверку if ! docker pull, если скачивание не удалось, то скрипт прерывает работу (exit 1) и не пытается запустить старую версию. Если все успешно, то после скачивания удаляется старый контейнер, если он есть. Далее запускаем контейнер с именем, который указан в параметрах и который будет собираться из скачанного образа, после чего выходим из докер хаб

```
        stage('Deploy') {
            steps {
                script {
                    echo "Развертывание на сервере ${params.server}..."

                    withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB', usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                        sshagent(credentials: ['ssh_key']) {
                            sh """
                            echo "Подключаемся к серверу и скачиваем образ с Docker Hub..."

                            ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null \\
                                little@${params.server} bash -s << 'EOF'
                                echo "Входим в Docker Hub на сервере..."
                                echo "${DH_PASS}" | docker login -u "${DH_USER}" --password-stdin

                                echo "Скачиваем образ ${DOCKER_IMAGE} с Docker Hub..."
                                if ! docker pull ${DOCKER_IMAGE}; then
                                    echo "ОШИБКА: Не удалось скачать образ с Docker Hub!"
                                    exit 1
                                fi

                                echo "Останавливаем и удаляем старый контейнер..."
                                docker stop ${params.APP_NAME} 2>/dev/null || true
                                docker rm ${params.APP_NAME} 2>/dev/null || true

                                echo "Запускаем приложение..."
                                docker run \\
                                    --name ${params.APP_NAME} \\
                                    ${DOCKER_IMAGE}

                                echo "✅ Приложение выполнило свою задачу и завершилось."
                                echo "Выходим из Docker Hub (очистка)..."
                                docker logout
EOF
                            """
                        }
                    }
                }
            }
        }
```

```
+ echo Подключаемся к серверу и скачиваем образ с Docker Hub...
Подключаемся к серверу и скачиваем образ с Docker Hub...
+ ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null little@62.60.246.108 bash -s
Warning: Permanently added '62.60.246.108' (ED25519) to the list of known hosts.
Входим в Docker Hub на сервере...
Login Succeeded

WARNING! Your credentials are stored unencrypted in '/home/little/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Скачиваем образ keshabyebye/maven-app:v01 с Docker Hub...
v01: Pulling from keshabyebye/maven-app
6fe36b853c88: Pulling fs layer
6fe36b853c88: Download complete
6fe36b853c88: Pull complete
Digest: sha256:65343e28608936470164a4fa6591169dbfe7d2238cc183919ee37998c5766841
Status: Downloaded newer image for keshabyebye/maven-app:v01
docker.io/keshabyebye/maven-app:v01
Останавливаем и удаляем старый контейнер (если есть)...
maven-app
maven-app
Запускаем приложение...
Hello World!
✅ Приложение выполнило свою задачу и завершилось.
Выходим из Docker Hub (очистка)...
Removing login credentials for https://index.docker.io/v1/
```

21. Последний этап, проводим проверку запуска приложения. Снова используем конструкцию try-catch, подключаемся по ssh к целевому хосту, проверяем статус контейнера, далее проверяем логи контейнера и ищем то что он должен был вывести в моем случае это “Hello World”. Если на этапе try что-то ломается, то используются действия из блока catch, то есть выводит сообщение об ошибке и помечает проверку ка нестабильную currentBuild.result

```
        stage('Verify') {
            steps {
                script {
                    try {
                        echo "Проверяем работу приложения..."
                        sshagent(credentials: ['ssh_key']) {
                            sh """
                            ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null \\
                                little@${params.server} bash -s << 'EOF'
                                sleep 5

                                echo "Статус контейнера:"
                                docker ps -a | grep ${params.APP_NAME} || exit 1

                                echo "Логи приложения:"
                                docker logs ${params.APP_NAME} || true

                                if docker logs ${params.APP_NAME} 2>&1 | grep -q "Hello World"; then
                                    echo "Приложение работает корректно!"
                                    exit 0
                                else
                                    echo "Приложение запущено, но вывод отличается"
                                    exit 0
                                fi
 EOF
                            """
                        }
                    } catch (Exception e) {
                        echo "Ошибка проверки: ${e.message}"
                        currentBuild.result = 'UNSTABLE'
                    }
                }
            }
        }
    }
```

```
Проверяем работу приложения...
[Pipeline] sshagent
[ssh-agent] Using credentials jenkins
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-krVZyxMNFk2w/agent.391341
SSH_AGENT_PID=391345
Running ssh-add (command line suppressed)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null little@62.60.246.108 bash -s
Warning: Permanently added '62.60.246.108' (ED25519) to the list of known hosts.
Статус контейнера:
5654d73777da   keshabyebye/maven-app:v01   "java -jar app.jar"   7 seconds ago   Exited (0) 6
seconds ago             maven-app
Логи приложения:
Hello World!
Приложение работает корректно!
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 391345 killed;
[ssh-agent] Stopped.
```

22. Использую конструкцию управления ошибками. Блок always: выполняется всегда, выводит статус и очищает рабочую директорию jenkins. Success: соответственно выводит просто сообщение. Failure: срабатывает если запущенный пайплайн упал на любом из этапов и отправляет уведомление о провальной сборки в чат вк (был создан ранее для выполнения ДЗ-29)

```
    post {
        always {
            echo "Пайплайн завершен. Статус: ${currentBuild.result ?: 'SUCCESS'}"
            cleanWs()
        }
        success {
            echo "Все этапы выполнены успешно!"
        }
        failure {
            script {
                echo "Пайплайн завершился с ошибками! Отправляю уведомление..."
                sh """
                MESSAGE="[ОШИБКА] Сборка \${JOB_NAME} #\${BUILD_NUMBER}! Ссылка: \${BUILD_URL}"
                curl -s -X POST "https://api.vk.com/method/messages.send" \\
                   -d "peer_id=366737789" \\
                   -d "message=\${MESSAGE}" \\
                   -d "random_id=\$(date +%s)" \\
                   -d "access_token=${VK_ACCESS_TOKEN}" \\
                   -d "v=5.199" > /dev/null
                echo "Уведомление отправлено."
                """
            }
        }
    }
}
```

```
Пайплайн завершен. Статус: SUCCESS
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] echo
Все этапы выполнены успешно!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```